

Document number	P3739R3
Date	2025-10-05
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Working Group (LWG)

Standard Library Hardening - using `std::optional<T&>`

Table of contents

- Standard Library Hardening - using `std::optional<T&>`
 - Changelog
 - Abstract
 - Motivation
 - Wording
 - Impact on the standard
 - Frequently Asked Questions
 - References

Changelog

R3

- added more context to historical discussion
- added Frequently Asked Questions section

R2

- added `Inspecting exception_ptr` ^[1] ^[2]
- added `define_static_{string,object,array}` ^[3]
- added more `const`

- added more formatting

Abstract

Utilize `std::optional<T&>` ^[4] and `T&` in the C++26 standard instead of `T*` order to harden it against null pointer dereference errors for similar memory safety reasons as Standard Library Hardening ^[5] and Minor additions to C++26 standard library hardening ^[6].

This was discussed during the March 2024 WG21 meeting in Tokyo, Japan at the Library Working Group during the standardization of `inplace_vector` ^[7] but could not be adopted since `std::optional<T&>` ^[4:1] was not sufficiently along in the C++26 standardization process. Now that `std::optional<T&>` ^[4:2] was adopted during the June 2025 Sophia meeting, it would be ideal that `inplace_vector` ^[8] and other superfluous instances of `T*` underwent minor revisions to better align it with the rest of C++26 and consequently provide a more robust interface.

The first request is that 3 of `inplace_vector`'s ^[8:1] modifiers be revised to return a `const std::optional<T&>` ^[4:3] instead of a pointer.

```
template< class... Args > constexpr pointerconst optional<T&> try_emplace_back(
Args&&... args );

constexpr pointerconst optional<T&> try_push_back( const T& value );

constexpr pointerconst optional<T&> try_push_back( T&& value );
```

The second request is that `Inspecting exception_ptr` ^[1:1] ^[2:1] be similarly revised.

```
template <class E> constexpr const E*const optional<const T&>
exception_ptr_cast(const exception_ptr& p) noexcept
```

The third request is that `define_static_{string,object,array}` ^[3:1] be similarly revised.

```
template <class T>
constexpr auto define_static_object(T&& v) -> remove_reference_t<T>const*const T&;
```

Motivation

Returning a pointer from these three methods are less than an ideal interface. The primary problem is that it leads to null pointer dereference errors. It is essentially a 100% unsafe interface. Now contrast that with the different iterations of `optional`.

While `optional<T>` can't be used as an alternative to pointer, in C++17 when it was standardized, `optional` had an equal number of unsafe and safe modifiers. The unsafe ones were `operator->` and `operator*` and the safe modifiers were `value` and `value_or`. In other words, `optional` was born with 50% null pointer dereference safety.

In C++23, the `and_then`, `transform` and `or_else` modifiers were added. This brought the total of modifiers to 5 of 7. The result is that 71% of all modifiers are safe from null pointer dereference errors.

However, only in C++26 with the advent of `std::optional<T&>` [4:4] does `optional` becomes a viable substitute for a nullable pointer. Also in this release is `std::optional` range support [9] which turns `optional` is a range of 0 or 1. This opens the door for 125 range algorithms and 34 range adapters to become null pointer deference safe modifiers for `optional`. Those range algorithms and adapters are growing at a much faster rate than new modifiers being added to the `optional` class itself. This brings the modifier safety rating up to 98%. The original 2 unsafe modifiers and hence the remaining 2% are being addressed by Standard Library Hardening [5:1].

Besides the safety benefit, the monadic and range usage are more ergonomic than the legacy pointer based interface.

There are equivalent benefits for `exception_ptr_cast` [1:2] [2:2] and `define_static_object` [3:2].

Wording

23.3.16 Class template `inplace_vector` [`inplace.vector`]

23.3.16.1 Overview [`inplace.vector.overview`]

...

(5.3.3) — If

...

```
namespace std {  
template<class T, size_t N>  
class inplace_vector {  
public:
```

...

```
template< class... Args >  
constexpr pointerconst optional<T&> try_emplace_back( Args&&... args );  
  
constexpr pointerconst optional<T&> try_push_back( const T& value );  
  
constexpr pointerconst optional<T&> try_push_back( T&& value );
```

...

```
};  
}
```

...

23.3.16.5 Modifiers [inplace.vector.modifiers]

...

```
template<class... Args>  
constexpr pointerconst optional<T&> try_emplace_back(Args&&... args);  
constexpr pointerconst optional<T&> try_push_back(const T& x);  
constexpr pointerconst optional<T&> try_push_back(T&& x);
```

⁸ Let `vals` denote a pack:

(8.1) — `std::forward<Args>(args)...` for the first overload,

(8.2) — `x` for the second overload,

(8.3) — `std::move(x)` for the third overload.

⁹ *Preconditions:* `value_type` is Cpp17EmplaceConstructible into `inplace_vector` from `vals....`

- ¹⁰ *Effects*: If `size() < capacity()` is true , appends an object of type `T` direct-non-list-initialized with `vals....` . Otherwise, there are no effects.
- ¹¹ *Returns*: `nullptr` if `size() == capacity()` is true , otherwise `addressof(back())` .
- ¹² *Throws*: Nothing unless an exception is thrown by the initialization of the inserted element.
- ¹³ *Complexity*: Constant.
- ¹⁴ *Remarks*: If an exception is thrown, there are no effects on `*this` .

Impact on the standard

Other than `inplace_vector` ^[8:2], `Inspecting_exception_ptr` ^[1:3] ^[2:3] and `define_static_{string,object,array}` ^[3:3] which are all three completely new to the standard, there are no other changes to the library standard and since `C++26` has not be released, this tweak is not expected to cause any problems.

The proposed changes are relative to the current working draft `N5008` ^[10].

Frequently Asked Questions

Wasn't the return type of `try_push_back` and `try_emplace_back` settled in R6 and R7?

According to `NB-Commenting is Not a Vehicle for Redesigning inplace_vector` , ^[11] R6 ^[12] discussed `optional<T>` , not `optional<T&>` . According to `NB-Commenting is Not a Vehicle for Redesigning inplace_vector` , ^[11:1] R7 ^[13] discussed `bool` and pointer but not `optional<T&>` . Using `optional<T&>` was discussed during the March 2024 WG21 meeting in Tokyo, Japan at the Library Working Group during the consideration of R11 ^[7:1]. It was known that `optional<&>` was coming at some point and when it does that it would need to be considered revising the return parameters of `try_push_back` and `try_emplace_back` . `optional<T&>` did come and it does need to be considered on both safety and on internal `C++26` consistency grounds.

Why `const <optional&>` ?

Either `const <optional&>` or `<optional&>` will mitigate the spatial memory errors such as null pointer dereference, pointer arithmetic and pointer indexing associated with superfluous pointer usage. So removing `const` doesn't negate the safety advantages of providing `<optional&>` . So why `const` ? We are talking about references which essentially are `const`

pointers. Consequently, `const optional&` should be the default usage when one is expecting reference semantics. Why do many programmers prefer reference semantics over pointer semantics! Besides the spatial safety benefits already mentioned, reference semantics also helps temporal safety because once a reference aliases an object with global, stack or heap lifetime it always does so, which makes it easier for programmers to reason about lifetimes at compile time. Due to the mutability of non `const` pointers, reasoning about lifetimes are runtime in nature. Further, no functionality is lost by having reference semantics because `const optional<>` is implicitly convertible to `optional<&>` just as `const` pointers are implicitly convertible to non `const` pointers.

```
int value = 0;
```

```
1  int * p00 = &value;
2  int const * p10 = &value;
3  int * const p01 = &value;
4  int const * const p11 = &value;
5  p00 = p00;
6  //p00 = p10;
7  p00 = p01;
8  //p00 = p11;
9  p10 = p00;
10 p10 = p10;
11 p10 = p01;
12 p10 = p11;
13 //p01 = p00;
14 //p01 = p10;
15 //p01 = p01;
16 //p01 = p11;
17 //p11 = p00;
18 //p11 = p10;
19 //p11 = p01;
20 //p11 = p11;
```

```
1  optional<int&> op00{value};
2  optional<const int&> op10{value}
3  const optional<int> op01{value}
4  const optional<const int> op11{
5  op00 = op00;
6  //op00 = op10;
7  op00 = op01;
8  //op00 = op11;
9  op10 = op00;
10 op10 = op10;
11 op10 = op01;
12 op10 = op11;
13 //op01 = op00;
14 //op01 = op10;
15 //op01 = op01;
16 //op01 = op11;
17 //op11 = op00;
18 //op11 = op10;
19 //op11 = op01;
20 //op11 = op11;
```

Doesn't `optional<T&>` have issues?

Sure and programmers are raising safety issues with `inplace_vector<T>`. National Body Comments is about fixing issues.

LWG4299 ^[14] and LWG4300 ^[15] are tiny clarifications. LWG4299 ^[14:1], LWG4300 ^[15:1] and LWG4300 ^[15:2] does not impact the public interface of `optional<&>` nor does it have safety impacts. Further these issues could be fixed in C++29 without it being a breaking change. Waiting till C++29 to correct the return type of `try_emplace_back` and `try_push_back` would be a breaking change, which is why it needs to be addressed now.

Isn't `optional<T&>` NOT trivially copyable?

That is true. While that should be fixed now, it is not a breaking change if it was fixed early in C++29. There is even a paper that requests this, *Make `optional<T&>` trivially copyable* ^[16]. What would be even better if the second design decision was chosen; "Additionally constrain its layout so it actually becomes a wrapper for a `T*`". ^[16:1] This could be done simply by removing "*// exposition only*" and by adding a constructor from a pointer to the `optional<&>` ^[4:5] proposal. This would allow programmers to opt into optional safety when working with legacy API(s) that are still superfluously using pointers, thus improving the safety benefits of C++26 further via minimal corrections.

Shouldn't `try_*_back` mostly only be used in boolean context?

According to *NB-Commenting is Not a Vehicle for Redesigning `inplace_vector`* ^[11:2], yes! `optional<&>` ^[4:6] was specifically designed to be used in a boolean context as seen by the multiple examples in the comparison table section at the start of that proposal.

Shouldn't we not be adding any more overhead to an object that is likely to be used briefly as a temporary and thrown away?

YES, that is why `optional<&>` ^[4:7] should eventually be made *trivially copyable* ^[16:2] and more important "additionally constrain its layout so it actually becomes a wrapper for a `T*`". ^[16:3]

Why `const T&` instead of `const T*` ?

If the referent is non null, as it should be when dealing with objects of both static storage duration and automatic storage duration, then a reference should be used instead of a pointer. Beside that, `const T&` is ready to be consumed as input arguments.

C++ Core Guidelines F.16: For "in" parameters, pass cheaply-copied types by value and others *by reference to const* ^[17]

We can have functions like `define_static_object` dereference the pointer a constant number of times, once per return, or we can have all programmers dereference the return every time the function is used as an input argument. In other words, a very large number that grows with time. The default choice seems clear.

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2927r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2927r3.html>) ↩ ↩ ↩ ↩
2. <https://wiki.edg.com/pub/Wg21sofia2025/StrawPolls/P3748R0.html> (<https://wiki.edg.com/pub/Wg21sofia2025/StrawPolls/P3748R0.html>) ↩ ↩ ↩ ↩
3. <https://wiki.edg.com/pub/Wg21sofia2025/StrawPolls/p3491r3.html> (<https://wiki.edg.com/pub/Wg21sofia2025/StrawPolls/p3491r3.html>) ↩ ↩ ↩ ↩
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf>) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩
5. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html>) ↩ ↩
6. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html>) ↩
7. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r11.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r11.html>) ↩ ↩
8. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r14.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r14.html>) ↩ ↩ ↩
9. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3168r2.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3168r2.html>) ↩
10. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>) ↩
11. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3830r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3830r0.pdf>) ↩ ↩ ↩

12. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0843r6.html>
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0843r6.html>) ↩
13. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0843r7.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0843r7.html>) ↩
14. <https://cplusplus.github.io/LWG/issue4299> (<https://cplusplus.github.io/LWG/issue4299>) ↩ ↩
15. <https://cplusplus.github.io/LWG/issue4300> (<https://cplusplus.github.io/LWG/issue4300>) ↩ ↩ ↩
16. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3836r0.html>
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3836r0.html>) ↩ ↩ ↩ ↩
17. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in>) ↩